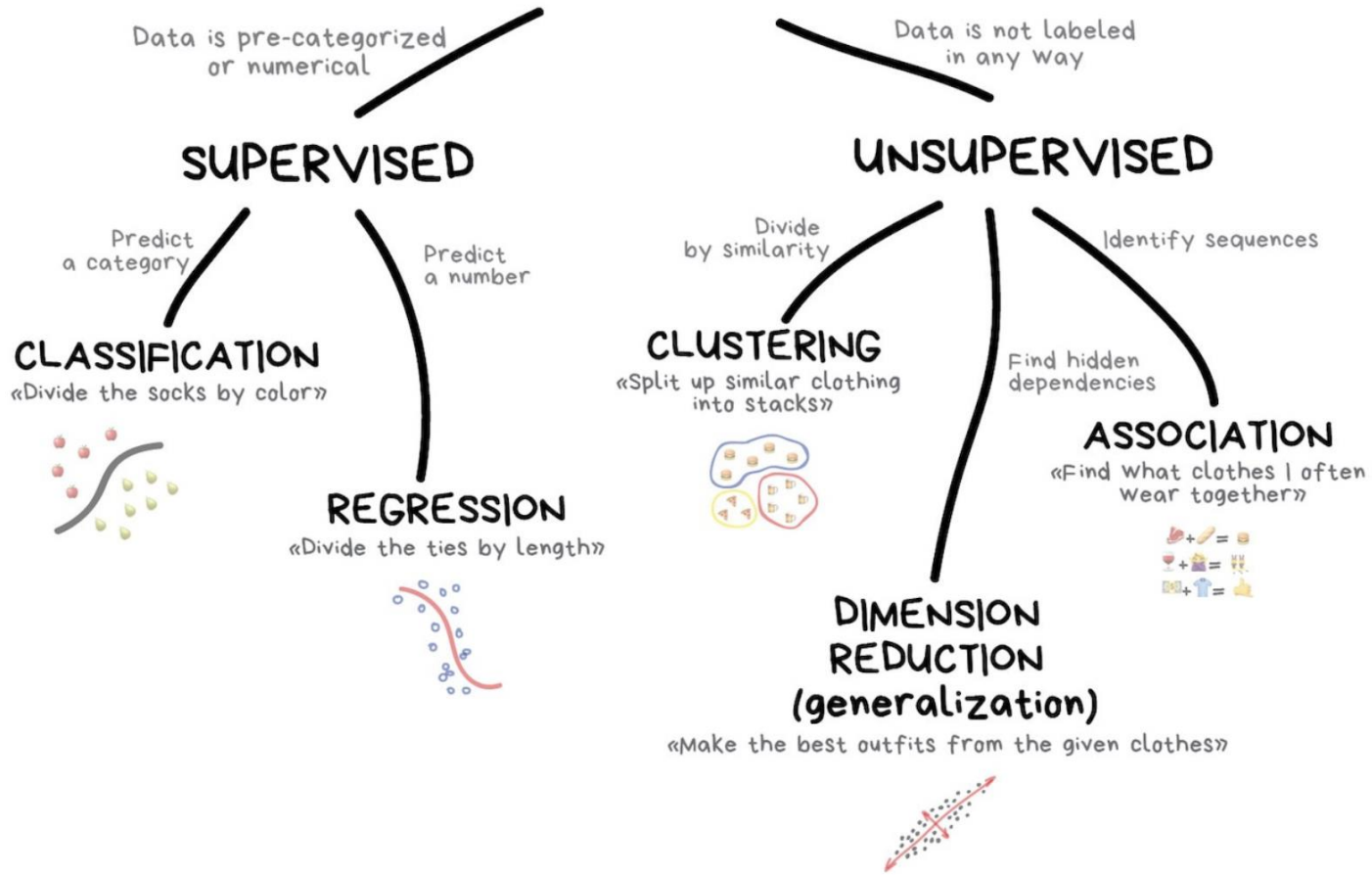
Data Classification: Logistic Regression & KNN

Assoc. Prof. Kitsana Waiyamai, Ph. D.

Dept. Of Computer Engineering
Faculty of Engineering, Kasetsart University
Bangkok, Thailand



Classical Machine Learning



https://vas3k.com/blog/machine_learning/?ref=hn

Types of Learning Methods

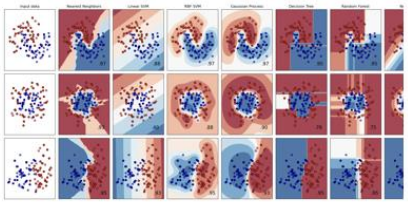
- Supervised learning
  - We have INPUT and desired OUTPUT
  - Learns relation between INPUT and OUTPUT
  - We know the classes (labels) => Labeled Data
  - We know the number of classes
- Un-supervised learning
  - We have only INPUT but no OUTPUT
  - We do not really know what we are looking for
  - We want to discover hidden or un-known patterns
  - We do not know the classes
    => Un-Labeled Data
  - We may not know the number of classes
- Semi-Supervised learning
  - Use both INPUT and OUTPUT
  - Use both labeled and un-labeled data
- Reinforcement learning
  - We have GOAL and available ACTION
  - We want a model (agent) to GOAL achievement using available ACTION without human supervision
  - Kind of Trial and Error
  - Reward is associated with the action
  - Data is created during the learning step

Scikit-Learn: https://scikit-learn.org

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.
Algorithms: Gradient boosting, nearest neighbors, random forest, logistic regression, and more...

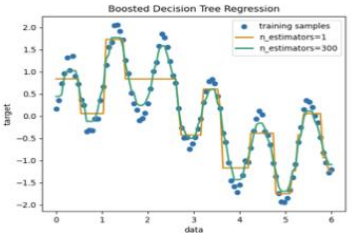


Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.
Algorithms: Gradient boosting, nearest neighbors, random forest, ridge, and more...

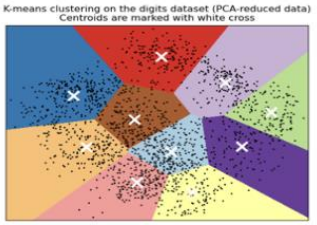


Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes
Algorithms: k-Means, HDBSCAN, hierarchical clustering, and more...

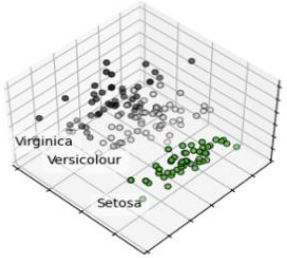


Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency
Algorithms: PCA, feature selection, non-negative matrix factorization, and more...

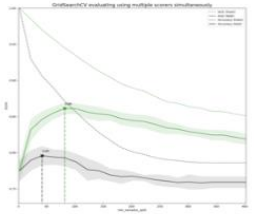


Examples

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning
Algorithms: grid search, cross validation, metrics, and more...

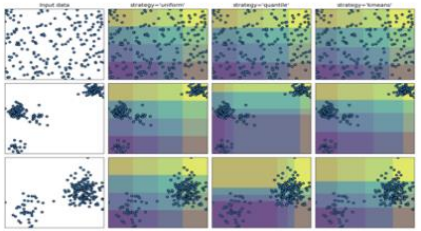


Examples

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.
Algorithms: preprocessing, feature extraction, and more...



Examples

ข้อมูล)

- Data classification is a supervised learning technique to predict category of new data based on training data
- Given a collection of records (*training set*)
  - Each record contains a set of *attributes*
  - One of the *class attribute (label)*

salary	*education*	label
10,000	high-school	reject
40,000	under-graduate	accept
15,000	under-graduate	reject
75,000	graduate	accept
18,000	graduate	accept

Training set

Data Classification: Definition

- Find a *model* to predict categorial class attribute as a function of the values of other attributes.
- Goal: Unseen records should be assigned a class as accurately as possible.
  - A *test set* is used to determine the accuracy of the model.
  - Usually, the given data set is divided into training and test sets, with training set used to build the model and test set used to validate it.

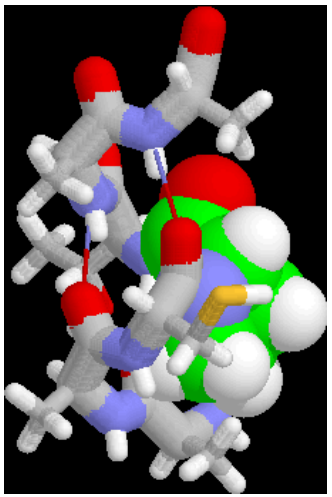
Examples of Classification Task

- Classify credit card transactions as legitimate or fraudulent.



- Categorize news stories as finance, weather, entertainment, sports, etc.

- Classify secondary structures of protein as alpha-helix, beta-sheet, or random coil.



Types of Data Classification:

- **Binary-classification (Binomial-classification)**
  - Number of classes is **two**.
  - Each data point belongs to exactly **one class**.
  - **Example**:
    - Classify emails as spam or not spam.
    - Predict whether a loan applicant will get approved or rejected.
- **Multiclass-classification**
  - Number of classes is **more than two**.
  - Each data point belongs to exactly **one class**.
  - **Example**:
    - Classify documents as topic "politics", "sports", or "other".
- **Multi-label classification**
  - Each data point can be associated with **multiple-classes**.
  - **Example**:
    - Tag images with multiple categories like "cat," "dog," and "beach".
    - Classify documents with relevant topics like "politics" and "sports".
- **Hierarchical classification**
  - Classes are organized in a **hierarchical structure**.
  - For example, classifying animals as mammals -> carnivores -> cats.
  - Allows models to capture relationships between classes.

ข้อมูล)

Binary Classification



- Spam
- Not spam

Multiclass Classification

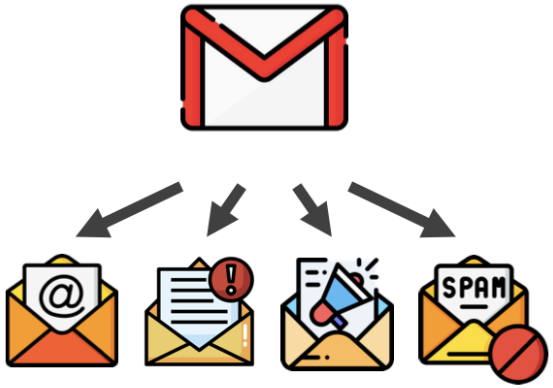


- Dog
- Cat
- Horse
- Fish
- Bird
- ...

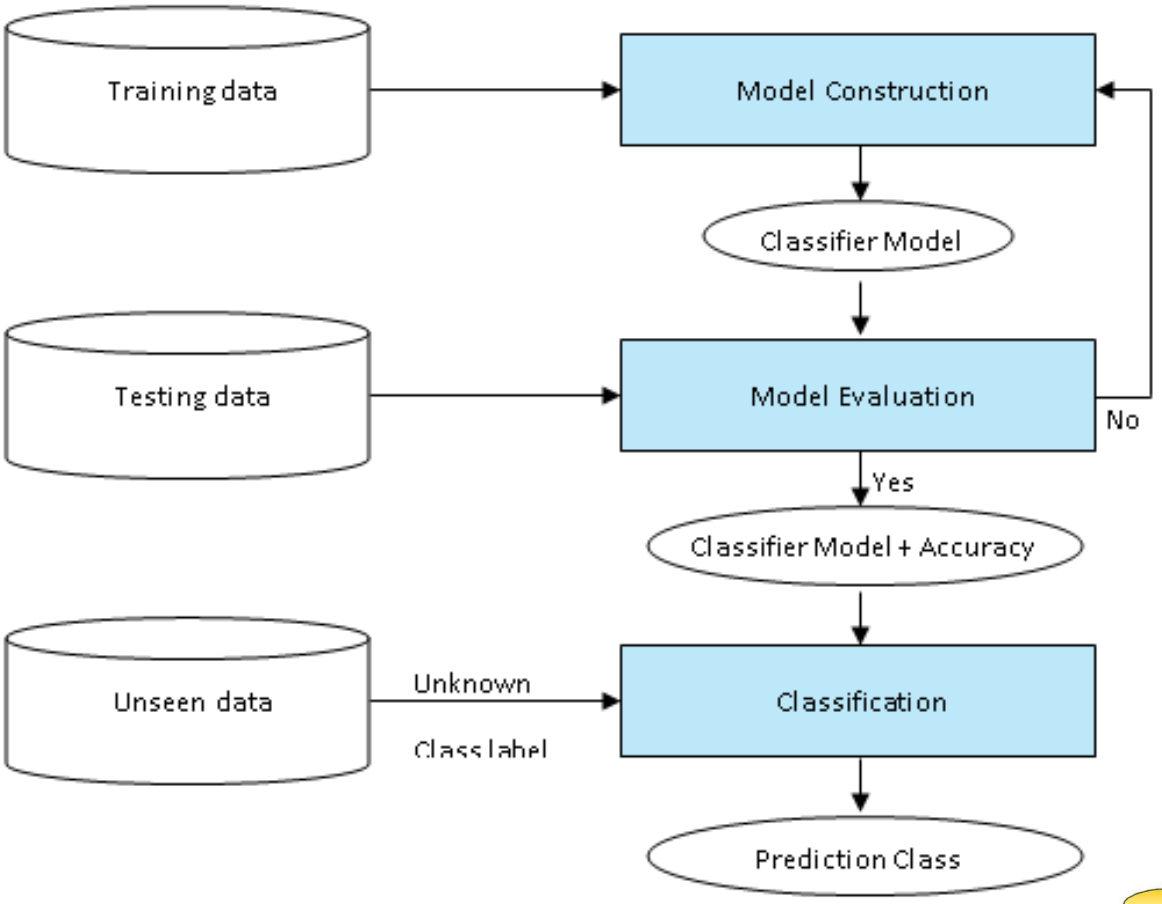
Multi-label Classification

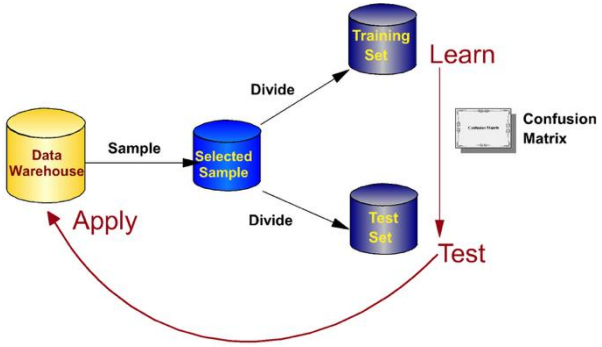


- Dog
- Cat
- Horse
- Fish
- Bird



Steps of Data Classification:





Python Classification Modelling Template

```
# Classification template

# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the dataset
dataset = pd.read_csv('Social_Network_Ads.csv')
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values

# Splitting the dataset into the Training set and Test set
from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)

# Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Fitting classifier to the Training set
# Create your classifier here

# Predicting the Test set results
y_pred = classifier.predict(X_test)

# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
```

Data Classification (การจำแนกประเภทข้อมูล)

- Classification models, both linear and non-linear, are used for predicting a categorial value
- Classification techniques:
  - Logistic Regression
  - K-Nearest Neighbors (K-NN)
  - Support Vector Machine (SVM)
  - Kernel SVM
  - Naive Bayes
  - Decision Tree Classification
  - Random Forest Classification
  - Etc.

An Overview of Classification Models

- Linear Models
  - Linear Regression
  - Regression Tree
- Rule-based Classification
  - PRISM
  - Decision List
  - Decision Trees
- Statistical Modeling
  - Naïve Bayes
- Etc.

- Instance-based Learning (Lazy Learning)
  - K-nearest-neighbor (KNN)
- Neural Networks
- Support Vector Machines (SVM)
- Genetic Algorithms
- Rough Set Theory & Fuzzy sets
- Etc.

สติก)

- Logistic regression predict a **categorical dependent variable** from a number of independent variables.

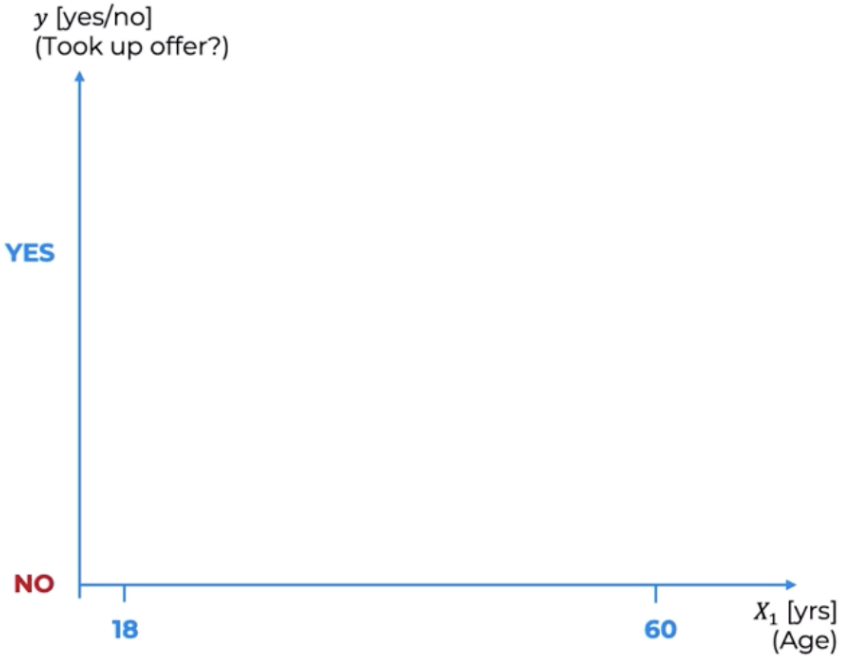


Will purchase health insurance: Yes / No

~



Age



สติก)

- Logistic regression predict a **categorical dependent variable** from a number of independent variables.
- Logistic regression is a **binary-classification** technique used to predict the probability of an observation belonging to one of two classes.

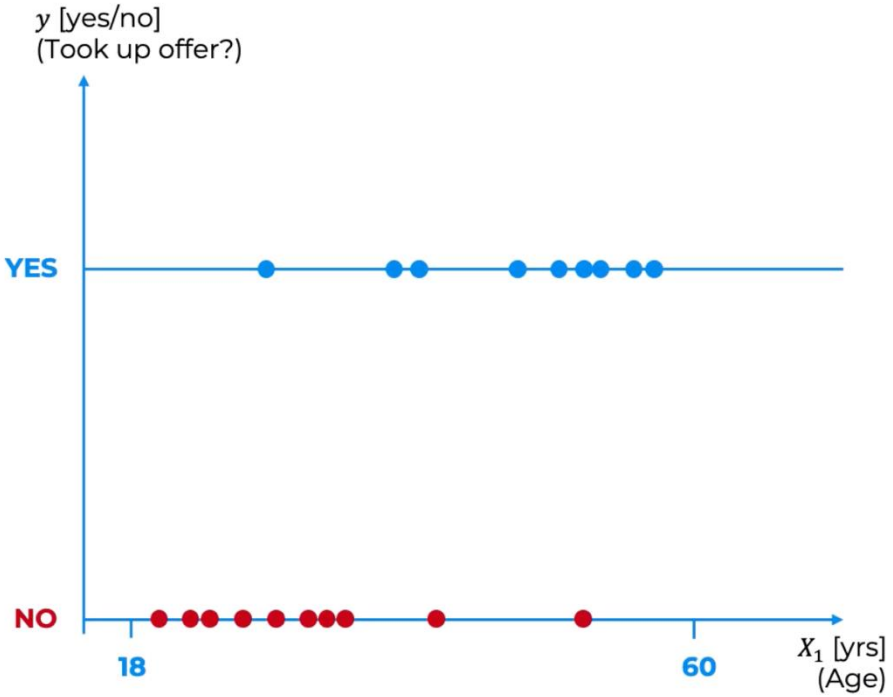


Will purchase health insurance: Yes / No

~



Age



สติก)

- Logistic regression predict a **categorical dependent variable** from a number of independent variables.
- Logistic regression is a **binary-classification** technique used to predict the probability of an observation belonging to one of two classes.
- User function similar to **sigmoid function** to predict outcome.



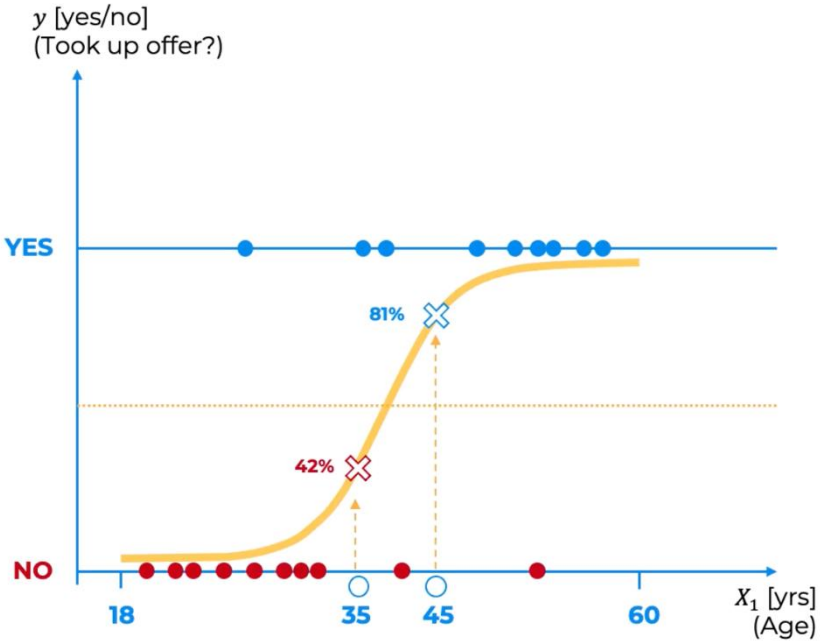
Will purchase health insurance: Yes / No

$\sim$



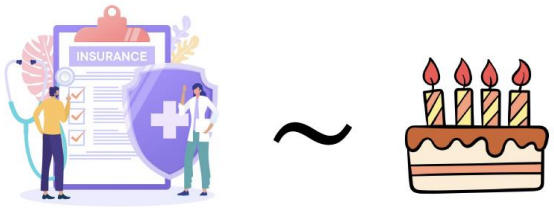
Age

$$\ln \frac{p}{1-p} = b_0 + b_1 X_1$$



สติก)

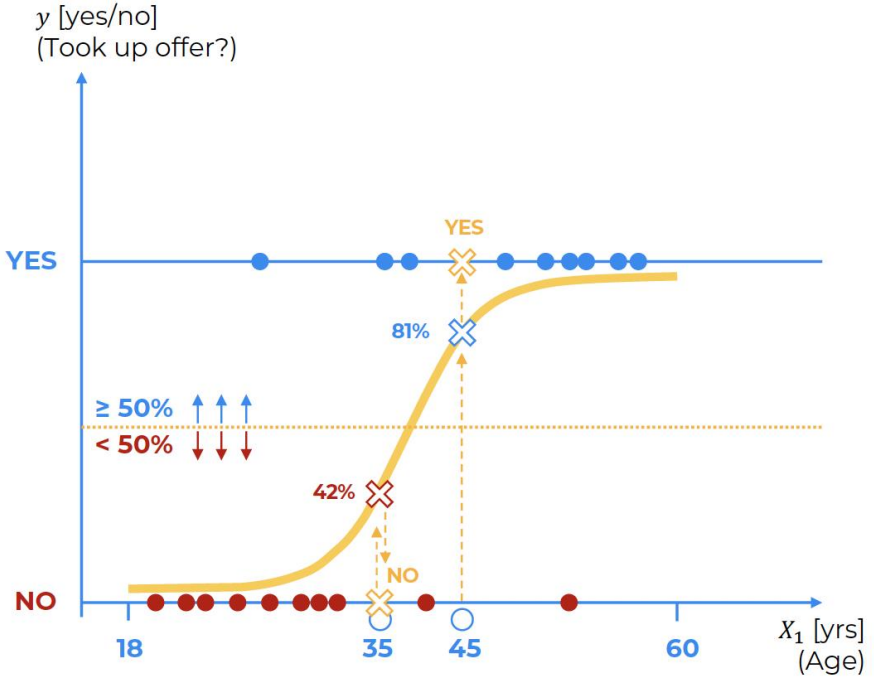
- Logistic regression predict a **categorical dependent variable** from a number of independent variables.
- Logistic regression is a **binary-classification** technique used to predict the probability of an observation belonging to one of two classes.
- User function similar to **sigmoid function** to predict outcome.



Will purchase health insurance: Yes / No

Age

$$\ln \frac{p}{1-p} = b_0 + b_1 X_1$$



สติก)

- Logistic regression predict a categorical dependent variable from a number of independent variables
- Logistic regression with **multiple variables**.



Will purchase health insurance: Yes / No

~



Age



Income



Level of Education

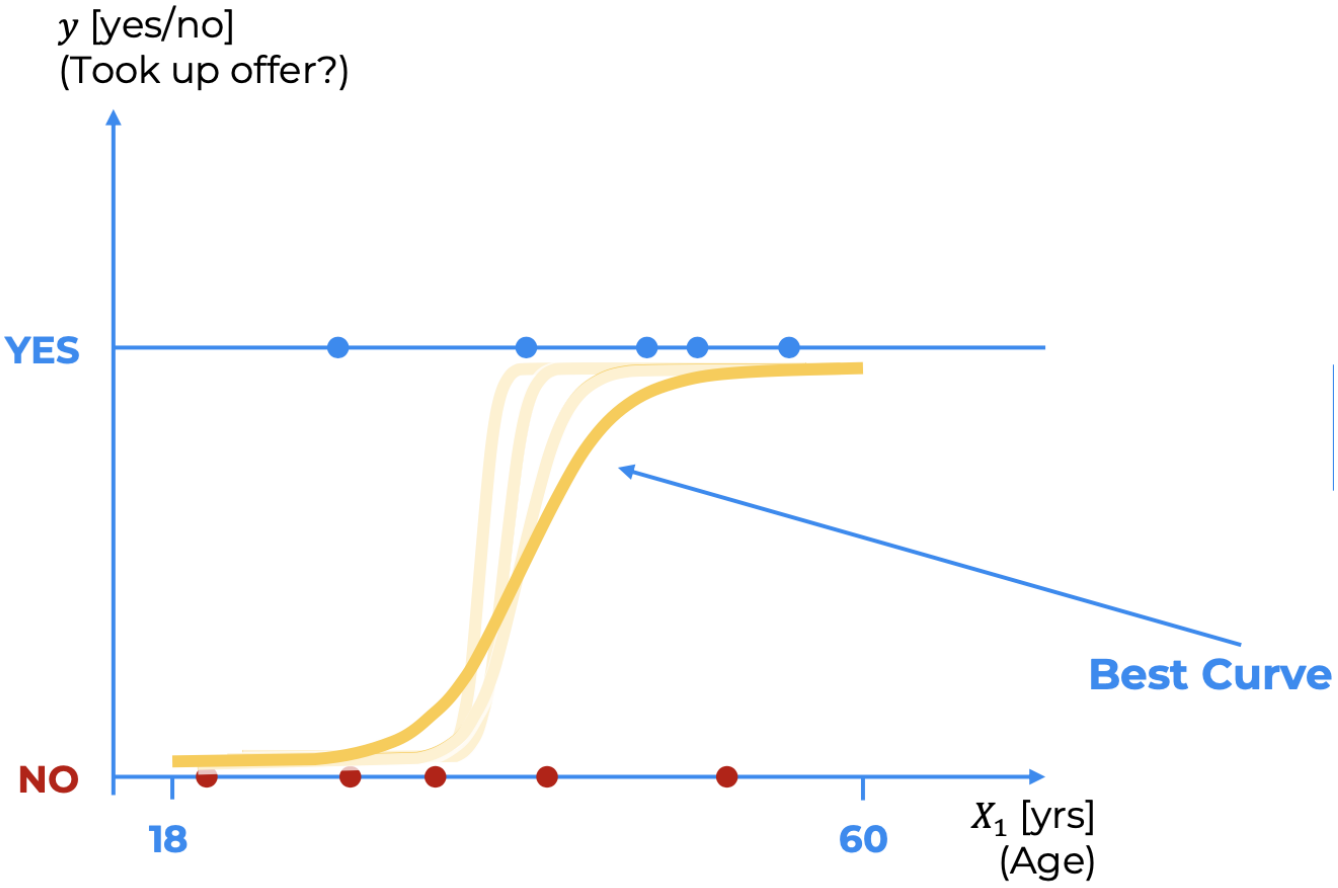


Family or Single

$$\ln \frac{p}{1-p} = b_0 + b_1X_1 + b_2X_2 + b_3X_3 + b_4X_4$$

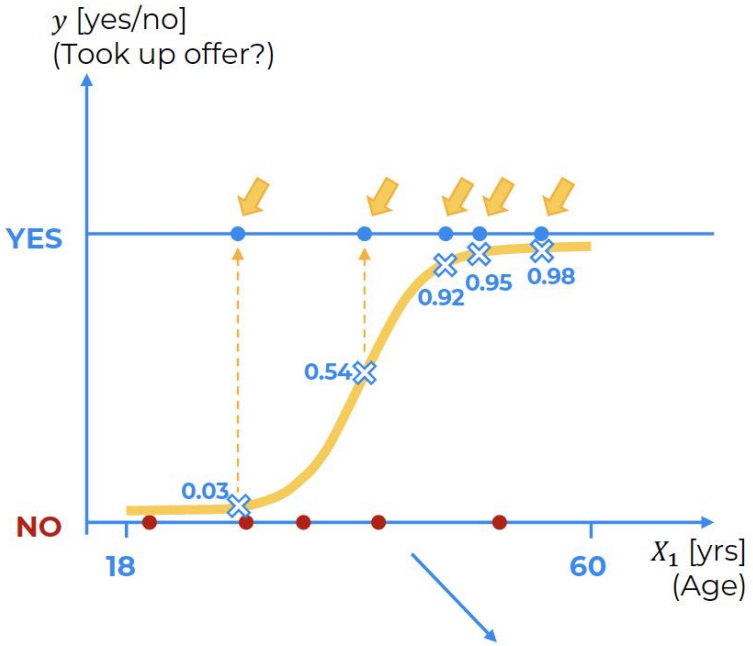
สติก)

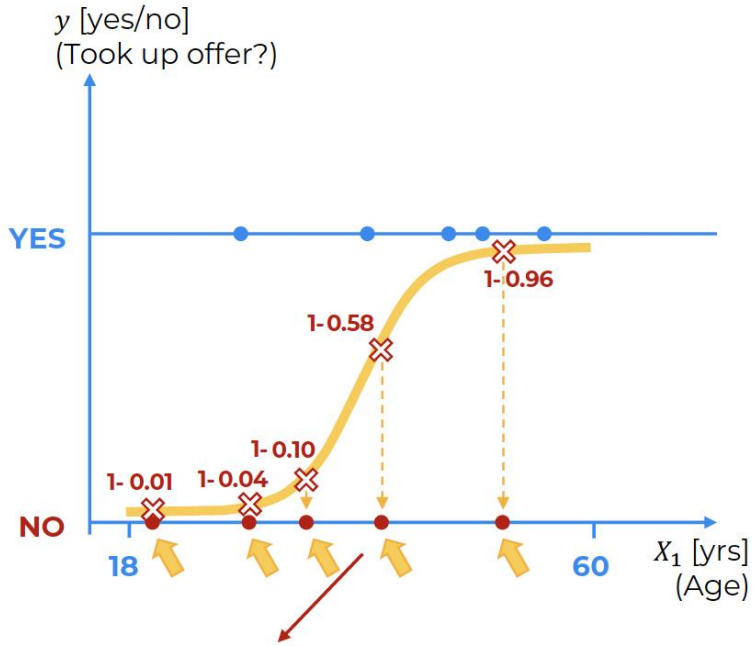
- Logistic regression predict a categorical dependent variable from a number of independent variables
- Logistic regression uses **Maximum Likelihood** to find the best line.



สติก)

- Logistic regression predict a categorical dependent variable from a number of independent variables
- Logistic regression uses **Maximum Likelihood** to find the best line.



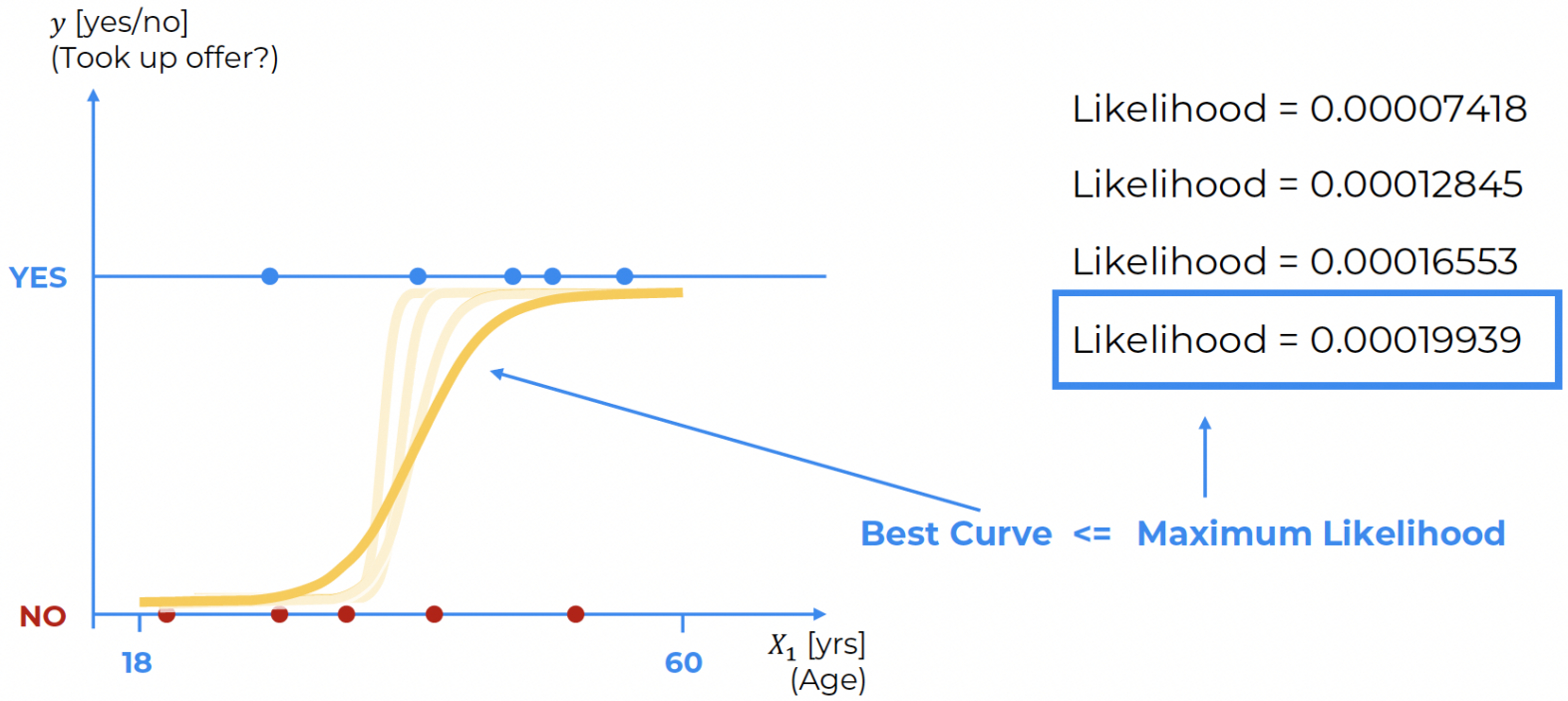


Likelihood = 0.03 x 0.54 x 0.92 x 0.95 x 0.98 x (1 – 0.01) x (1 – 0.04) x (1 – 0.10) x (1 – 0.58) x (1 – 0.96)

Likelihood = **0.00019939**

สติก)

- Logistic regression predict a categorical dependent variable from a number of independent variables
- Logistic regression uses **Maximum Likelihood** to find the best line.



Logistic Regression with Python

Importing the libraries

```
[ ]  1 import numpy as np
     2 import matplotlib.pyplot as plt
     3 import pandas as pd
     4 from sklearn.linear_model import LogisticRegression
```

Importing the dataset

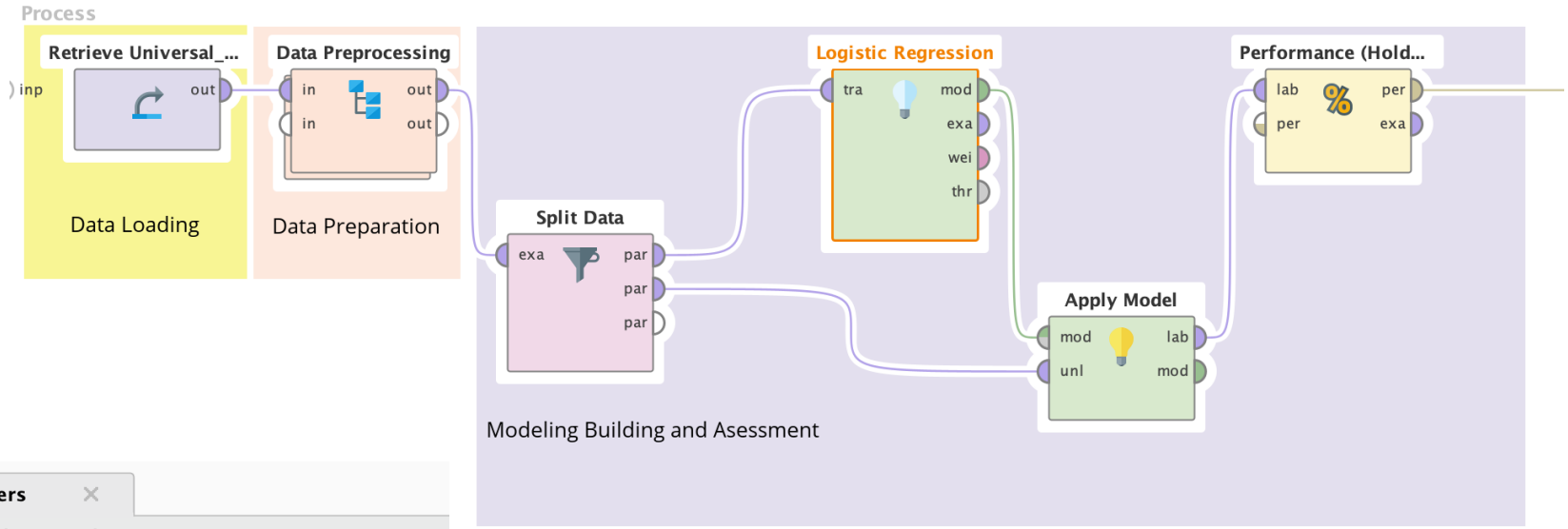
```
[ ]  1 df = pd.read_csv('simple-veh-class.csv')
```

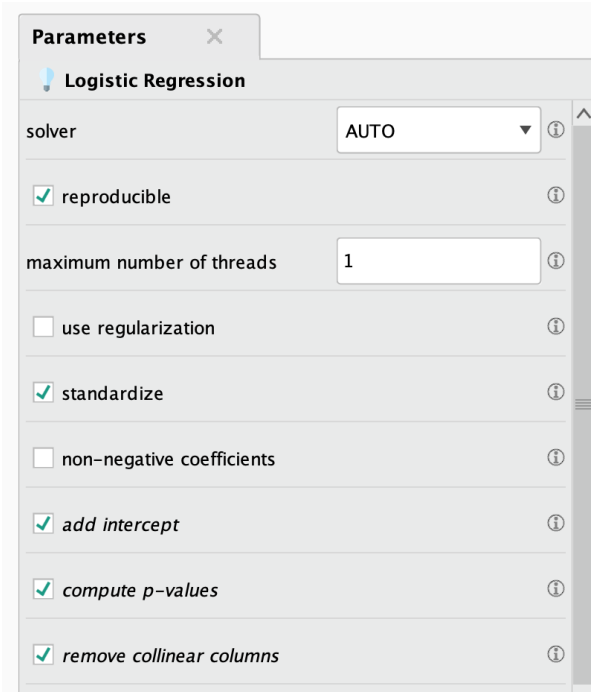
```
     1 X = df[['weight_kg', 'height_m', 'n_wheels']]
     2 y = df['vtype']
```

```
[ ]  1 model = LogisticRegression(solver = 'lbfgs')
     2 model.fit(X,y)
     3
     4 print("classes", model.classes_)
     5 print("features:", X.columns.tolist())
     6 print("coef:", model.coef_)
     7 print("intercept:", model.intercept_)
```

```
classes ['car' 'other']
features: ['weight_kg', 'height_m', 'n_wheels']
coef: [[5.11087515e-04 2.55647725e-01 6.77689408e-01]]
intercept: [-3.84078827]
```

Logistic Regression with RapidMiner





สติก)

- **Advantages:**
  - **Interpretability**: Easy to understand and interpret results.
  - **Simplicity**: Relatively simple and easy-to-implement algorithm.
  - **Performance**: Logistic Regression often performs well for binary classification tasks, especially when the data is **linearly separable**.
- **Limitations:**
  - **Assumption of linearity:** Assumes a **linear relationship** between the features and the target variable.
  - **Limited to binary classification:** Is primarily designed for **binary classification tasks**. There is extension to **multi-class classification tasks.**
  - **Susceptibility to overfitting:** In situations of high-dimensional features, Logistic Regression can be prone to **overfitting.**

K-nearest-neighbor (KNN) เทคนิคการค้นหาเพื่อนบ้านที่ใกล้ที่สุด k ตัว

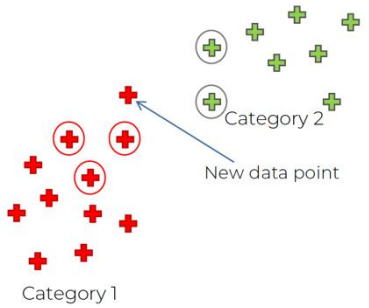
- K-Nearest Neighbor (KNN) is a supervised learning technique used for both classification and regression tasks.
- KNN works based on the principle of "birds of a feather flock together."
  นกประเภทเดียวกันมักบินรวมฝูง
- Data points with similar characteristics tend to belong to the same class.
- KNN determines the **most frequent class** among the **K nearest neighbors**. This class is then assigned to the new data point.





K-KNN more explanation

1. **Define K**: The first step involves setting the K value, which represents the number of closest neighbors to consider for prediction. This crucial parameter significantly impacts the model's performance.
2. **Calculate Distances**: KNN calculates the distance between the new data point and all existing data points in the training set. Common distance metrics include **Euclidean distance**, Manhattan distance, and cosine similarity.
3. **Identify Nearest Neighbors**: Based on the calculated distances, KNN identifies the K data points closest to the new data point. These are considered the "nearest neighbors."
4. **Make Predictions**:
   - **For classification tasks**, KNN determines the **most frequent class** among the K nearest neighbors. This class is then assigned to the new data point.
   - **For regression tasks**, KNN calculates the **average value of the K nearest neighbors' target variables**. This value is then assigned to the new data point.



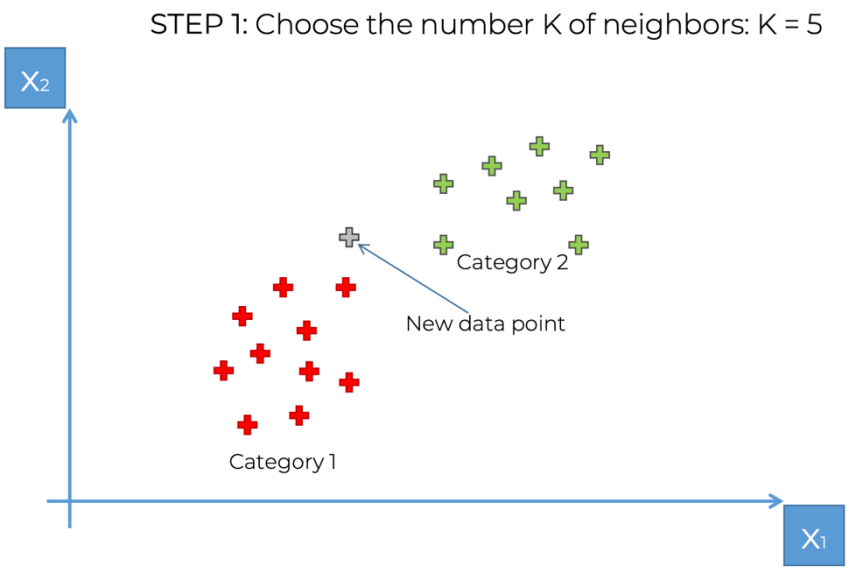
K-nearest-neighbor (KNN)

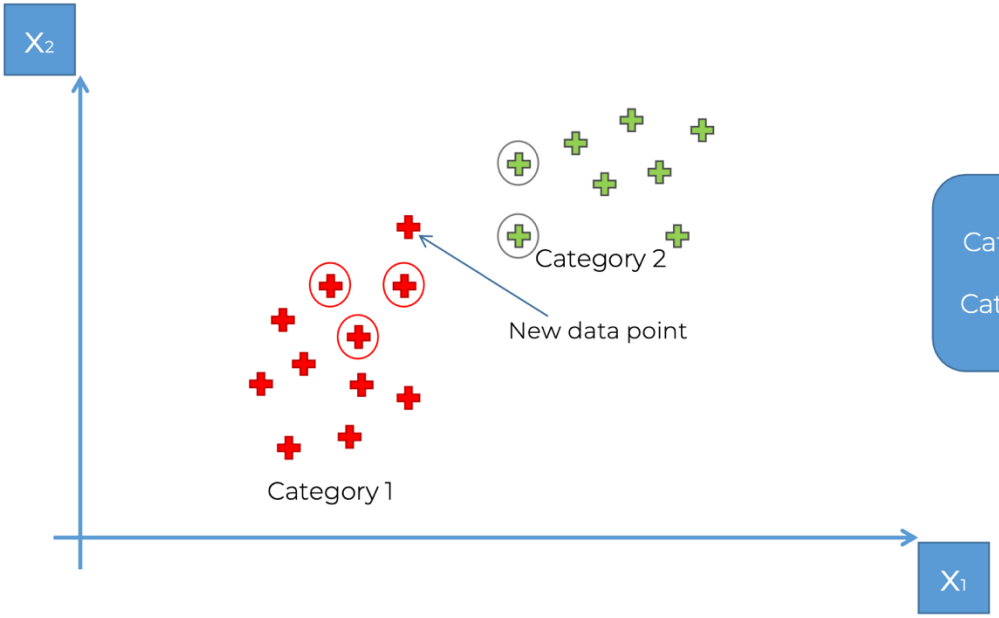
- For a given record to be classified, identify nearby records
- "Near" means records with similar predictor values $X_1, X_2, \ldots X_p$
- Classify the record as whatever the predominant class is among the nearby records (the "neighbors")



K-nearest-neighbor (KNN)

- Step1: Choose K (Number of Neighbors)
- Step2: Take the K Nearest Neighbors of the New Point, according to the distance
- Step3: Among these K Neighbors, Count the number of points in each class
- Step4: Assign the New Point to the Class having the most Neighbors

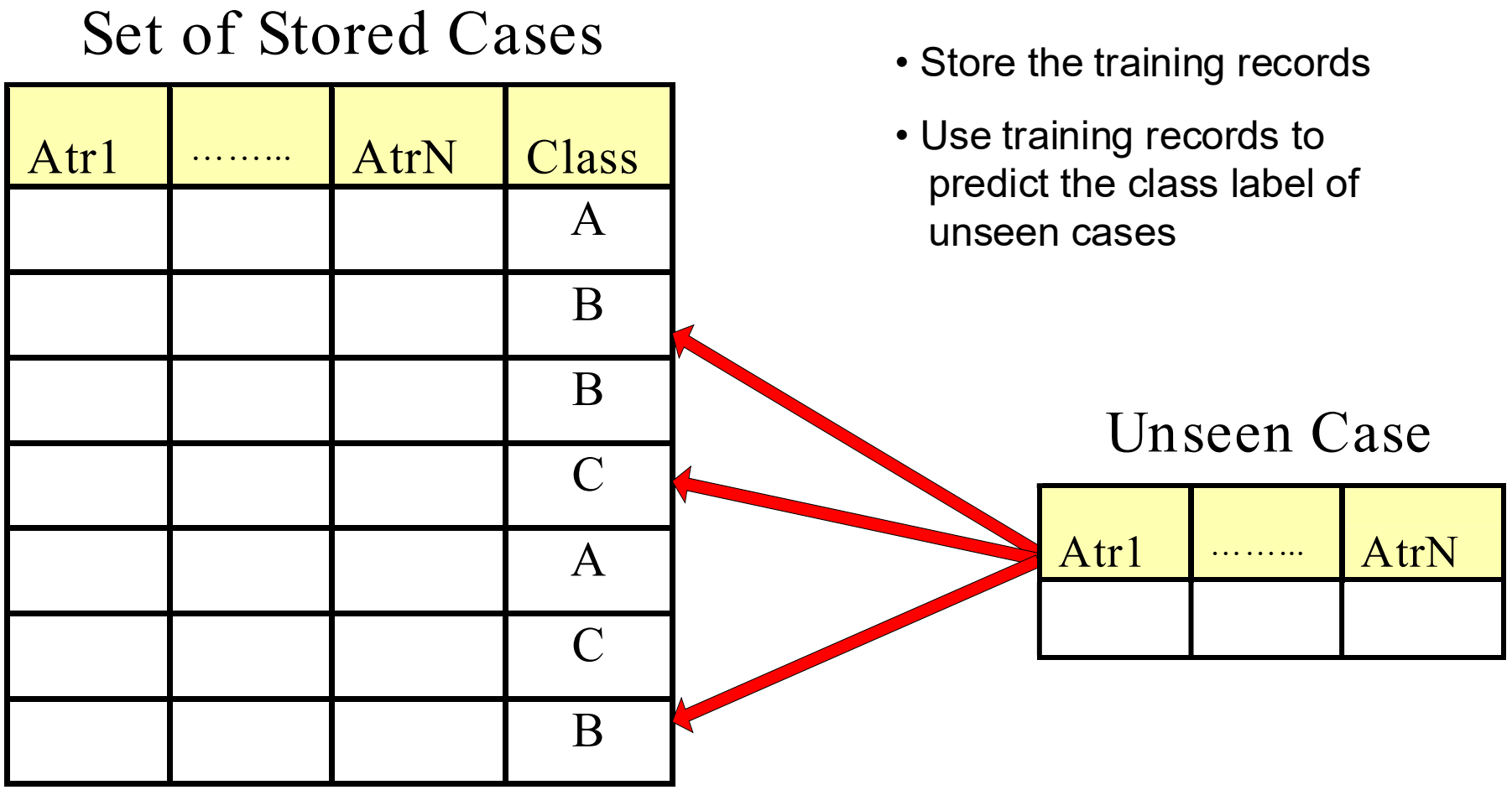




Category 1: 3 neighbors

Category 2: 2 neighbors

Instance-Based Classifiers



Instance-Based Classifiers

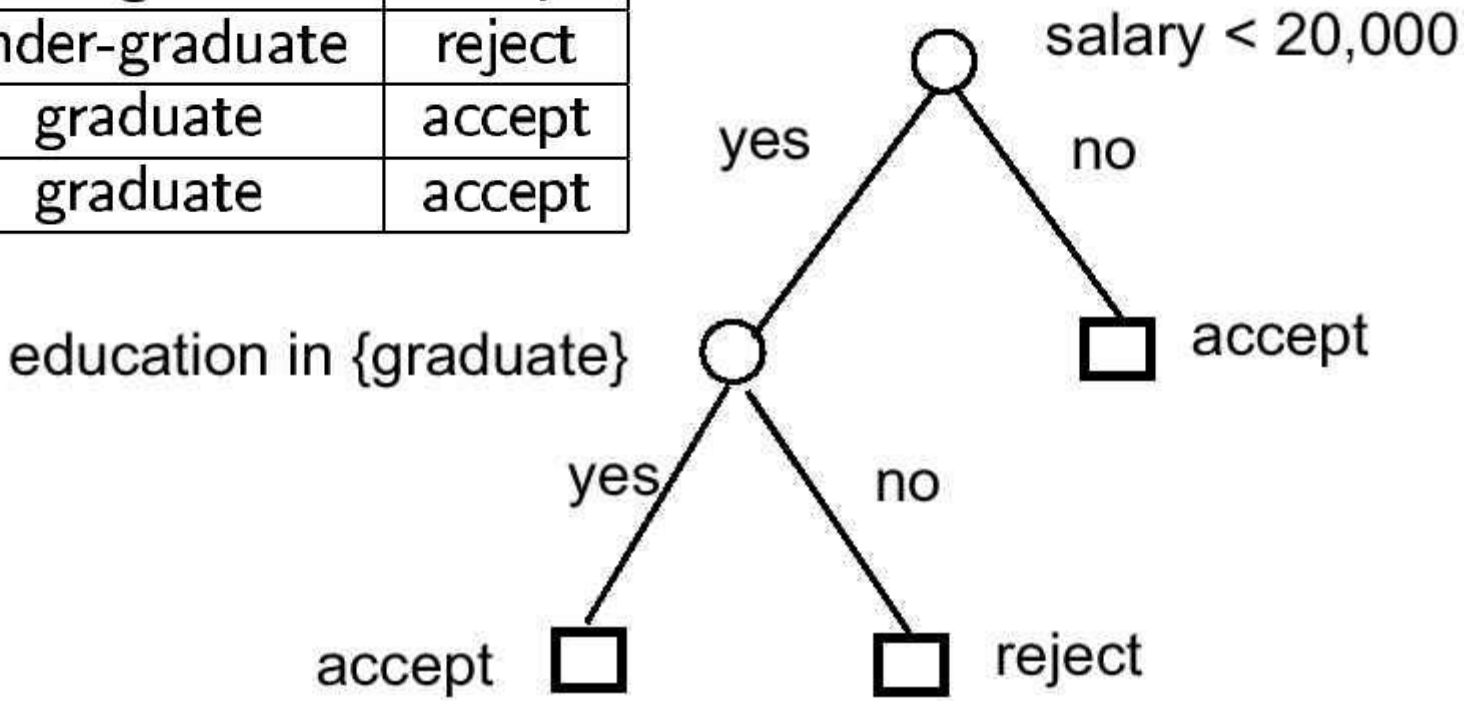
- Memorizes entire training data and performs classification only if attributes of record match one of the training examples exactly
- Nearest neighbor
  - Uses k “closest” points (nearest neighbors) for performing classification

Characteristics

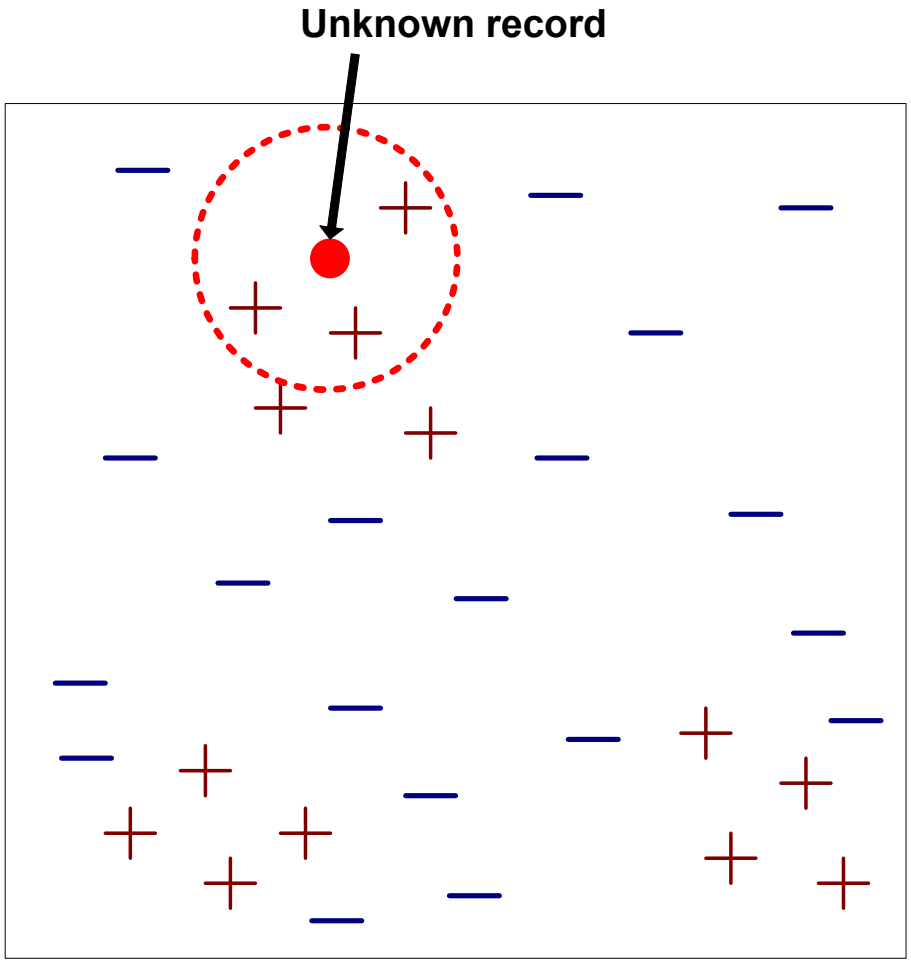
- Makes no assumptions about the data
- Does not build models explicitly
  - Data-driven, not model-driven
  - It is a lazy learning method
- Unlike eager learners such as decision tree induction and rule-based systems
- Classifying unknown records are relatively expensive

Decision Tree : Build models Explicitly

salary	*education*	label
10,000	high-school	reject
40,000	under-graduate	accept
15,000	under-graduate	reject
75,000	graduate	accept
18,000	graduate	accept



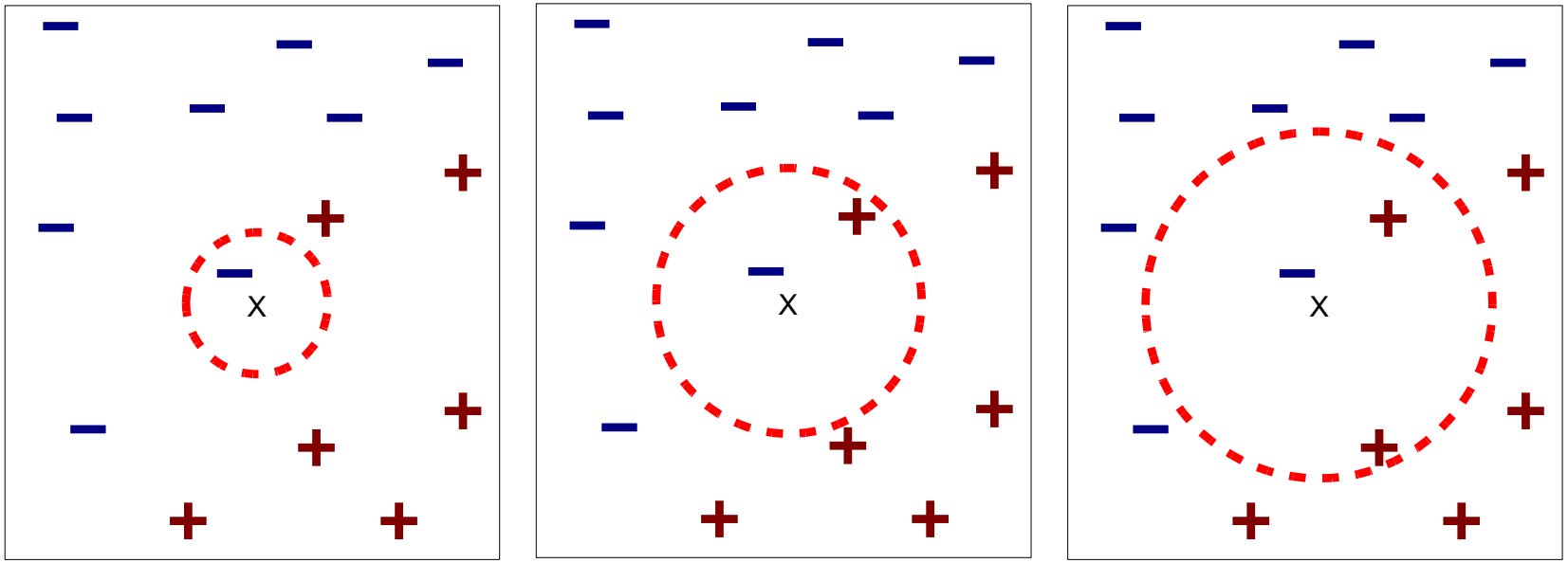
Nearest-Neighbor Classifiers



- Requires three things
  - The set of stored records
  - Distance Metric to compute distance between records
  - The value of *k*, the number of nearest neighbors to retrieve

- To classify an unknown record:
  - Compute distance to other training records
  - Identify *k* nearest neighbors
  - Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)

Definition of Nearest Neighbor



(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

K-nearest neighbors of a record x are data points that have the k smallest distance to x

How to measure “nearby”?

The most popular distance measure is **Euclidean distance**

$$\sqrt{(x_1 - u_1)^2 + (x_2 - u_2)^2 + \cdots + \left(x_p - u_p\right)^2}$$

Distance between Objects with Continuous Variables

- *Minkowski distance* :

  $d(i,j) = (\ |x_{i1} - x_{j1}|^q + |x_{i2} - x_{jp}|^q + \ldots + |x_{ip} - x_{jp}|^q)^{1/q}$

  where $i = (x_{i1}, x_{i2}, \ldots, x_{ip})$ and $j = (x_{j1}, x_{j2}, \ldots, x_{jp})$ are two p-dimensional data objects, and q is a positive integer.

- *Manhattan distance* : $q = 1$

  $d(i,j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{jp}| + \ldots + |x_{ip} - x_{jp}|$

- *Euclidean distance* : $q = 2$

  $d(i,j) = (\ |x_{i1} - x_{j1}|^2 + |x_{i2} - x_{jp}|^2 + \ldots + |x_{ip} - x_{jp}|^q)^{1/2}$

- Weighting !!! with *Euclidean distance*

  $d(i,j) = (w_1|x_{i1} - x_{j1}|^2 + w_2|x_{i2} - x_{jp}|^2 + \ldots + w_p|x_{ip} - x_{jp}|^q)^{1/2}$

Choosing k

K is the number of nearby neighbors to be used to classify the new record

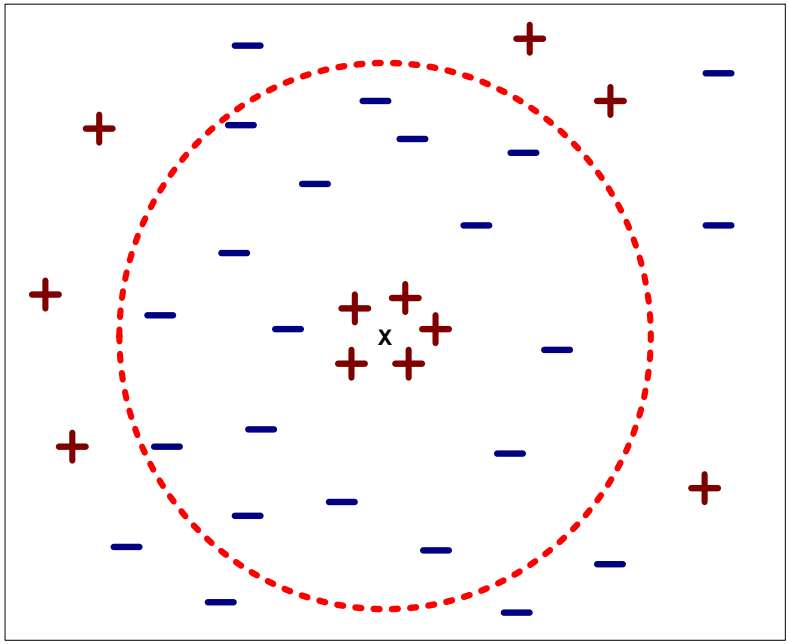
- *K*=1 means use the single nearest record
- *K*=5 means use the 5 nearest records

Typically choose that value of *k* which has lowest error rate in **validation data**

Typically choose Odd value for *k*

Nearest Neighbor Classification…

- Choosing the value of k:
  - If k is too small, sensitive to noise points
  - If k is too large, neighborhood may include points from other classes



Low *k* vs. High *k*

Low values of *k* (1, 3, …) capture local structure in data (but also noise)

High values of *k* provide more smoothing, less noise, but may miss local structure

Note: the extreme case of k = n (i.e., the entire data set) is the same as the "naïve rule" (classify all records according to majority class)

Example: Riding Mowers

Data: 24 households classified as owning or not owning riding mowers

Predictors: Income, Lot Size

Income	Lot_Size	Ownership
60.0	18.4	owner
85.5	16.8	owner
64.8	21.6	owner
61.5	20.8	owner
87.0	23.6	owner
110.1	19.2	owner
108.0	17.6	owner
82.8	22.4	owner
69.0	20.0	owner
93.0	20.8	owner
51.0	22.0	owner
81.0	20.0	owner
75.0	19.6	non-owner
52.8	20.8	non-owner
64.8	17.2	non-owner
43.2	20.4	non-owner
84.0	17.6	non-owner
49.2	17.6	non-owner
59.4	16.0	non-owner
66.0	18.4	non-owner
47.4	16.4	non-owner
33.0	18.8	non-owner
51.0	14.0	non-owner
63.0	14.8	non-owner

Example, XLMiner Output

For each record in validation data (6 records) XLMiner finds neighbors amongst training data (18 records).

The record is scored for k=1, k=2, … k=18.

Best k appears to be k=8.

k = 9, k = 10, k=14 also share low error rate, but best to choose lowest k.

Value of k	% Error Training	% Error Validation	
1	0.00	33.33	
2	16.67	33.33	
3	11.11	33.33	
4	22.22	33.33	
5	11.11	33.33	
6	27.78	33.33	
7	22.22	33.33	
8	22.22	16.67	<--- Best k
9	22.22	16.67	
10	22.22	16.67	
11	16.67	33.33	
12	16.67	16.67	
13	11.11	33.33	
14	11.11	16.67	
15	5.56	33.33	
16	16.67	33.33	
17	11.11	33.33	
18	50.00	50.00	

Using K-NN for Classification & Prediction

- Instead of “majority vote determines class” use average of response values
- May be a weighted average, weight decreasing with distance

Attributes may have to be scaled …

- Scaling issues
  - Attributes may have to be scaled to prevent distance measures from being dominated by one of the attributes
  - Example:
    - height of a person may vary from 1.5m to 1.8m
    - weight of a person may vary from 90lb to 300lb
    - income of a person may vary from $10K to $1M

Shortcomings

- Expected distance to nearest neighbor increases with *p* (with large vector of predictors, all records end up "far away" from each other)
- In a large training set,
  - It takes a long time to find distances to all the neighbors and then identify the nearest one(s)
- These constitute "curse of dimensionality"
- Reduce dimension of predictors (e.g., with PCA)
- Need to find Optimal K

Summary

- Find distance between record-to-be-classified and all other records
- Select k-nearest records
  - Classify it according to majority vote of nearest neighbors
  - Or, for prediction, take the as average of the nearest neighbors
- “Curse of dimensionality” – need to limit # of predictors
- KNN offers a powerful approach to classification and regression tasks

Advantages of KNN

- **KNN key advantages include:**
  - **Simplicity**: KNN is easy to understand and implement.
  - **No-assumptions required** about Linearity, Normal distribution, etc.
  - **Effective at capturing complex interactions** among variables without having to define a statistical model.
  - **Multi-Class classification** support.
  - **Flexibility**: The algorithm can handle different data types, including numerical and categorical features, without complex transformations.
  - **Interpretability**: The reasoning behind predictions is directly linked to the chosen neighbors, allowing for easier understanding.
  - **No Model Training**: KNN does not require explicit model training, making it computationally efficient for tasks where data updates are frequent.

Dis-advantages of KNN

- **While KNN offers numerous advantages, be aware of its limitations:**
  - **Sensitivity to Outliers**: Outliers in the training data can significantly impact KNN's performance, leading to inaccurate predictions.
  - **Curse of Dimensionality**: As the number of features increases, the effectiveness of KNN can decline due to the difficulty in finding truly "nearest" neighbors in a high-dimensional space.
  - **Computational Cost**: In large datasets, calculating distances to all training points for each new data point can be computationally expensive.

K-NN with Python

```python
# K-Nearest Neighbors (K-NN)

# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the dataset
dataset = pd.read_csv('Social_Network_Ads.csv')
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values

# Splitting the dataset into the Training set and Test set
from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)

# Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Fitting K-NN to the Training set
from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p = 2)
classifier.fit(X_train, y_train)

# Predicting the Test set results
y_pred = classifier.predict(X_test)

# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
```

K-NN with R

```r
# K-Nearest Neighbors (K-NN)

# Importing the dataset
dataset = read.csv('Social_Network_Ads.csv')
dataset = dataset[3:5]

# Encoding the target feature as factor
dataset$Purchased = factor(dataset$Purchased, levels = c(0, 1))

# Splitting the dataset into the Training set and Test set
# install.packages('caTools')
library(caTools)
set.seed(123)
split = sample.split(dataset$Purchased, SplitRatio = 0.75)
training_set = subset(dataset, split == TRUE)
test_set = subset(dataset, split == FALSE)

# Feature Scaling
training_set[-3] = scale(training_set[-3])
test_set[-3] = scale(test_set[-3])

# Fitting K-NN to the Training set and Predicting the Test set results
library(class)
y_pred = knn(train = training_set[, -3],
             test = test_set[, -3],
             cl = training_set[, 3],
             k = 5,
             prob = TRUE)

# Making the Confusion Matrix
cm = table(test_set[, 3], y_pred)
```

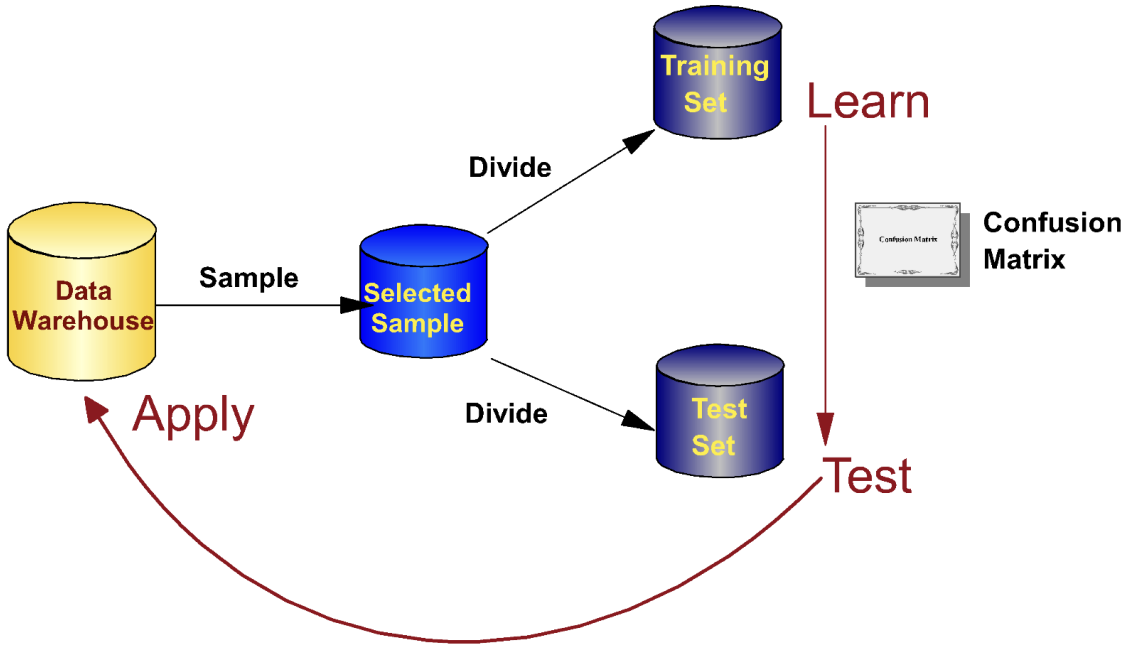
Pre-processing for supervised learning

- **Hold-out** divides the dataset into two parts.
  - First part is larger and is used as a learning dataset
    => Training-Set
  - Second part is test data set to measure the performance of the model.
    => Test-Set
  - Hold-out is used when there is a large amount of data.
  - Therefore, the characteristics and distribution of Training and Test Sets are similar.

- **Other methods**

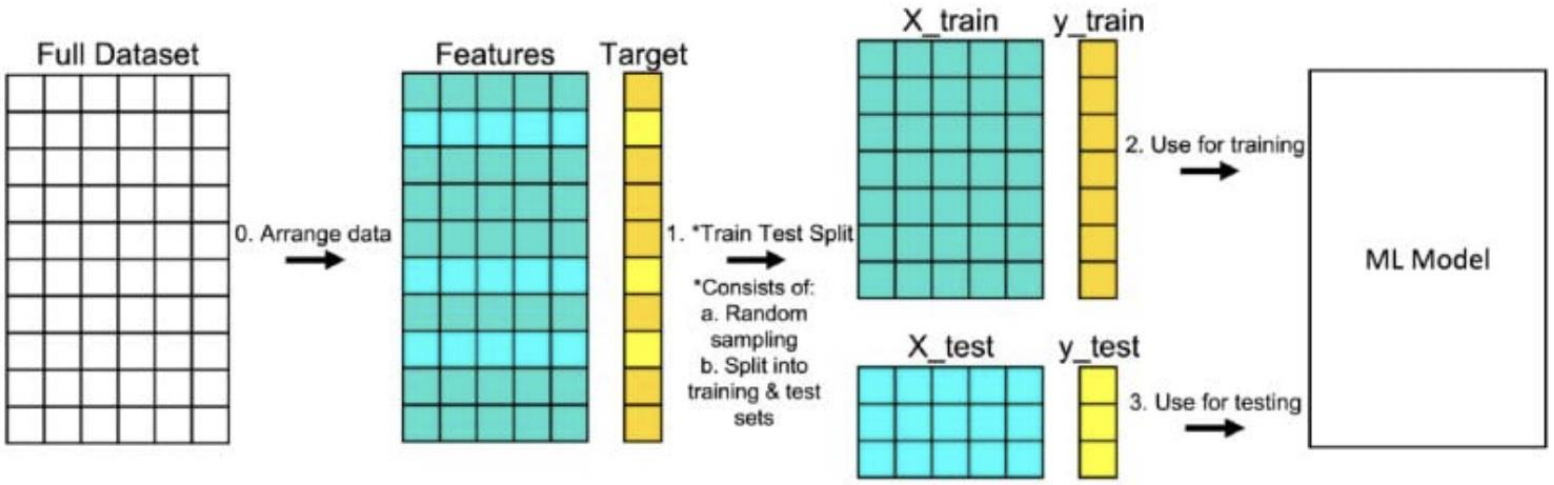
Hold-out divides the dataset into two parts

- Usually, the given data set is divided into training and test set, with training set used to build the model and test set used to validate it.
- A *test set* is used to determine the accuracy of the model.



Python train_test_split

Splitting the dataset into the Training set and Test set



Simple Dataset

Splitting the dataset into the Training set (80%) and Test set (20%)

Row No.	Age	Salary	ID	Country	Purchased
1	44	72000	1	France	No
2	27	48000	2	Spain	Yes
3	30	54000	3	Germany	No
4	38	61000	4	Spain	No
5	40	63778	5	Germany	Yes
6	35	58000	6	France	Yes
7	39	52000	7	Spain	No
8	48	79000	8	France	Yes
9	50	83000	9	Germany	No
10	37	67000	10	France	Yes

ID	Purchased	Country = France	Country = Spain	Country = Germany	Age	Salary
1	No	1	0	0	44	72000
2	Yes	0	1	0	27	48000
3	No	0	0	1	30	54000
4	No	0	1	0	38	61000
5	Yes	0	0	1	40	63778
6	Yes	1	0	0	35	58000
7	No	0	1	0	39	52000
8	Yes	1	0	0	48	79000
9	No	0	0	1	50	83000
10	Yes	1	0	0	37	67000

Python train_test_split operator

Splitting the dataset into the Training set (80%) and Test set (20%)

```
[ ] from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 1)
```

```
[ ] print(X_train)
```

```
[[0.0 0.0 1.0 38.77777777777778 52000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 35.0 58000.0]]
```

```
[ ] print(X_test)
```

```
[[0.0 1.0 0.0 30.0 54000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

```
[ ] print(y_train)
```

```
[0 1 0 0 1 1 0 1]
```

```
[ ] print(y_test)
```

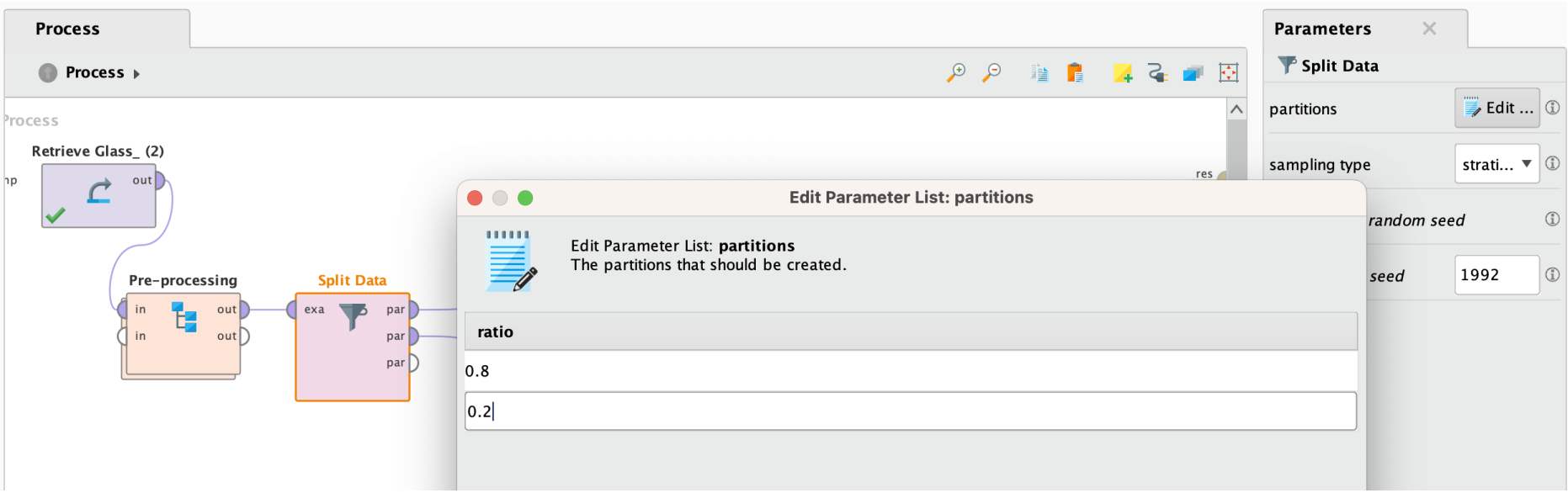
```
[0 1]
```

```
[ ] print(X)
```

```
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

RapidMiner split operator

Splitting the dataset into the Training set (80%) and Test set (20%)



Python StandardScaler operator

Feature Scaling on Training Set & Test Set

```
[ ]  from sklearn.preprocessing import StandardScaler
     sc = StandardScaler()
     X_train[:, 3:] = sc.fit_transform(X_train[:, 3:])
     X_test[:, 3:] = sc.transform(X_test[:, 3:])
```

```
[ ]  print(X_train)
```

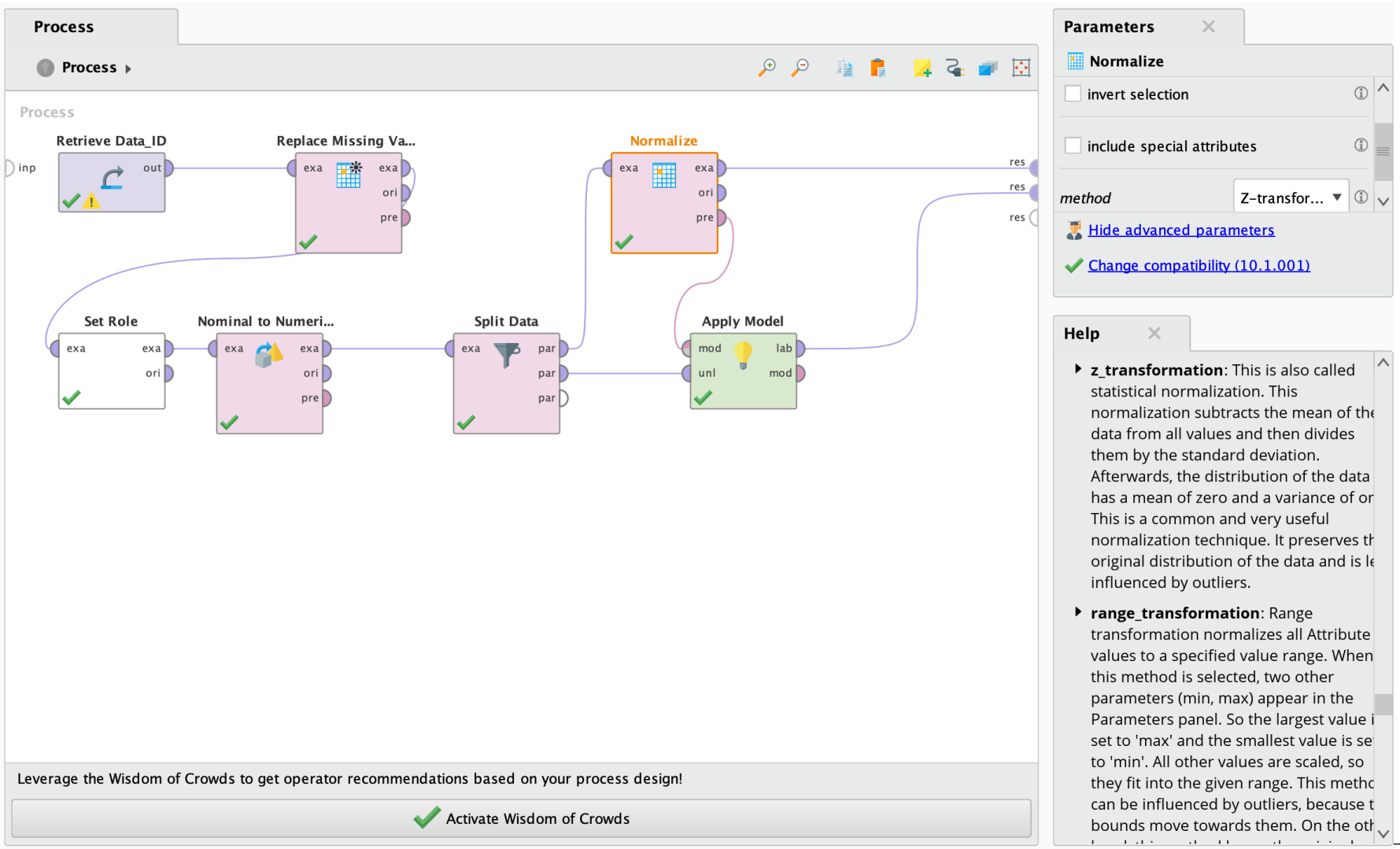
```
[[0.0 0.0 1.0 -0.19159184384578545 -1.0781259408412425]
 [0.0 1.0 0.0 -0.014117293757057777 -0.07013167641635372]
 [1.0 0.0 0.0 0.566708506533324 0.633562432710455]
 [0.0 0.0 1.0 -0.30453019390224867 -0.30786617274297867]
 [0.0 0.0 1.0 -1.9018011447007988 -1.420463615551582]
 [1.0 0.0 0.0 1.1475343068237058 1.232653363453549]
 [0.0 1.0 0.0 1.4379472069688968 1.5749910381638885]
 [1.0 0.0 0.0 -0.7401495441200351 -0.5646194287757332]]
```

```
[ ]  print(X_test)
```

```
[[0.0 1.0 0.0 -1.4661817944830124 -0.9069571034860727]
 [1.0 0.0 0.0 -0.44973664397484414 0.2056403393225306]]
```

RapidMiner Normalize operator

Feature Scaling on Training set and Test set



Data Leakage Problem?

- Data leakage occurs when information from outside the training dataset is used to create the model.
- This happens when data that should only be in the test set is included in the training set.

Problem Example: Normalization is performed using statistics (mean, variance) computed from **the entire dataset before splitting it into train and test sets** => The test set indirectly influences training.

Always split the dataset into training and test sets first, then apply preprocessing separately.

Measure for evaluating classification models

- Focus on the predictive capability of a model
  - Rather than how fast it takes to classify or build models, scalability, etc.

- Accuracy,
- Precision,
- Recall,
- F-Measure,
- AUC-ROC Curve
- Etc.

Confusion Matrix

- Confusion matrix visualizes the performance of a classification model by summarizing the number of correct and number incorrect predictions for each class.
- This one 2-class Confusion Matrix

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes(1)	a	b
	Class=No(0)	c	d

a: TP (true positive)

b: FN (false negative)

c: FP (false positive)

d: TN (true negative)

Confusion Matrix

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	a (TP)	b (FN)
	Class=No	c (FP)	d (TN)

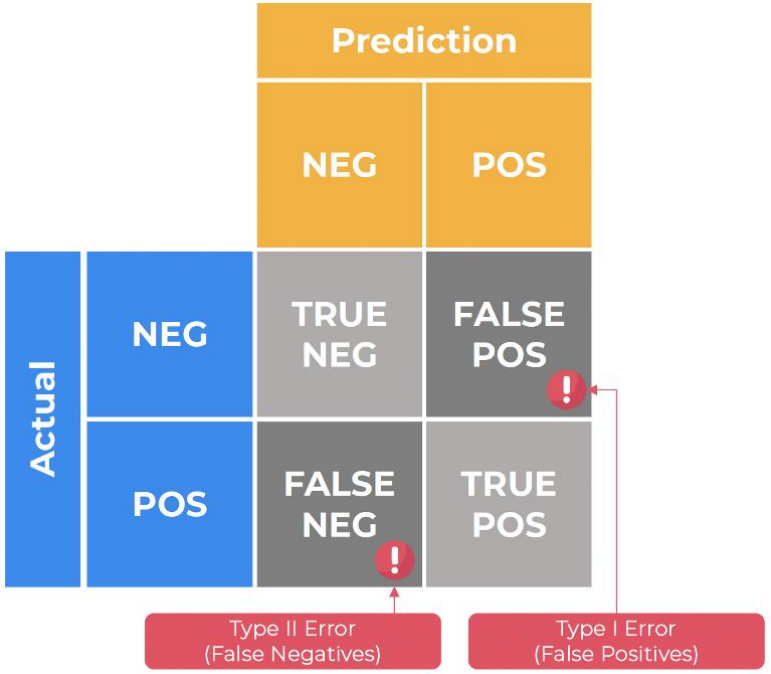
Confusion Matrix

Sometimes you may see “predicted” in rows, and “actual” in columns.

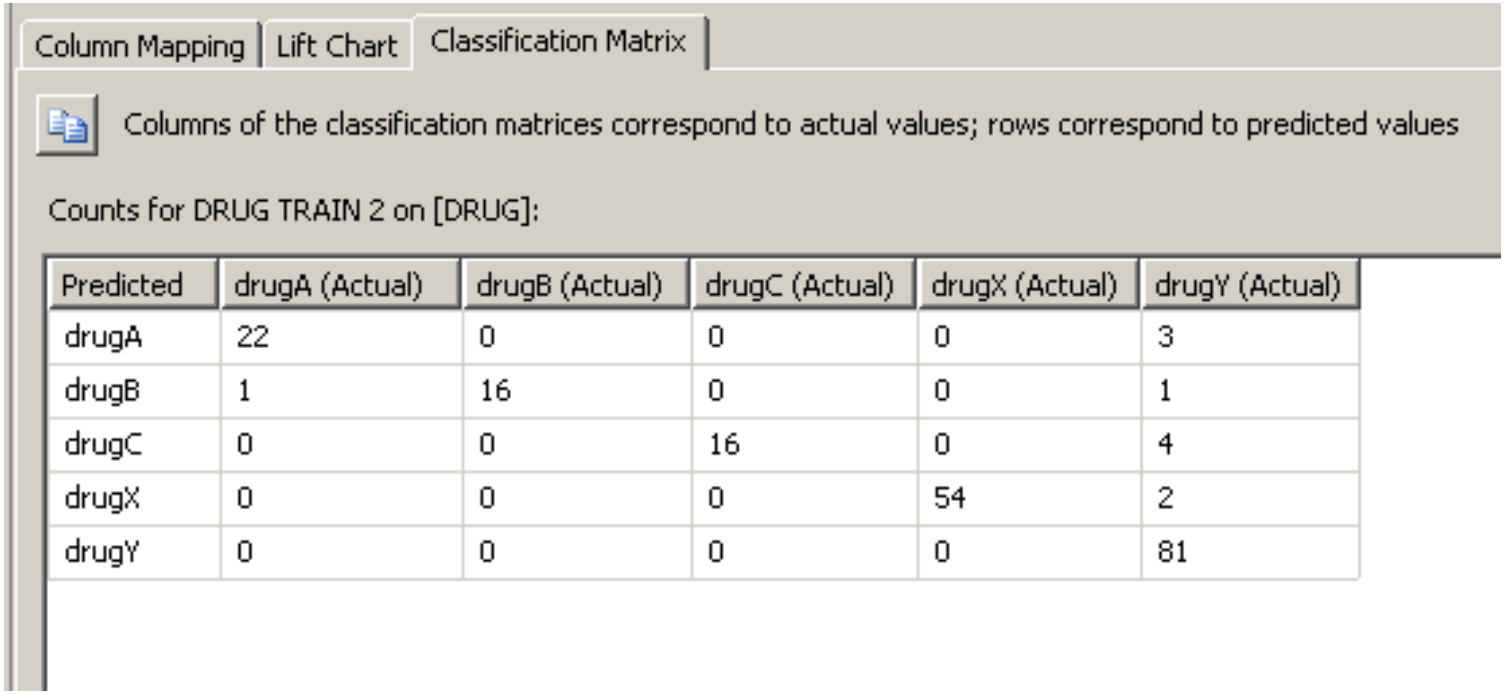
		Actual Class		
		Class 0	***Class 1 (positive class)***	Total Predicted (down)
Predicted Class	***Class 0***	**TN** (true negatives) # of class 0 cases correctly classified	**FN** (false negatives**, false dismissal**) # of class 1 cases incorrectly classified	TN + FN # of negative predictions
	Class 1 (positive class)	**FP** (false positives, **false alarm**) # of class 0 cases incorrectly classified	**TP** (true positives) # of class 1 cases correctly classified	TP + FP # of positive predictions
	Total Actual (across)	TN + FP # of actual negatives, i.e., class 0	TP + FN # of actual positives, i.e., class 1	TP + FN + FP + TN (total data)

Misclassification error

- Error = classifying a record as belonging to one class when it belongs to another class.
- Error rate = percent of misclassified records out of the total records in the test data.



Multi-Class Confusion Matrix


Column Mapping | Lift Chart | Classification Matrix

Columns of the classification matrices correspond to actual values; rows correspond to predicted values

Counts for DRUG TRAIN 2 on [DRUG]:

Predicted	drugA (Actual)	drugB (Actual)	drugC (Actual)	drugX (Actual)	drugY (Actual)
drugA	22	0	0	0	3
drugB	1	16	0	0	1
drugC	0	0	16	0	4
drugX	0	0	0	54	2
drugY	0	0	0	0	81

Accuracy

		PREDICTED CLASS	
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	a (TP)	b (FN)
	Class=No	c (FP)	d (TN)

$$\text{Accuracy} = \frac{a+d}{a+b+c+d} = \frac{TP+TN}{TP+TN+FP+FN}$$

Threshold (cutoff) for classification

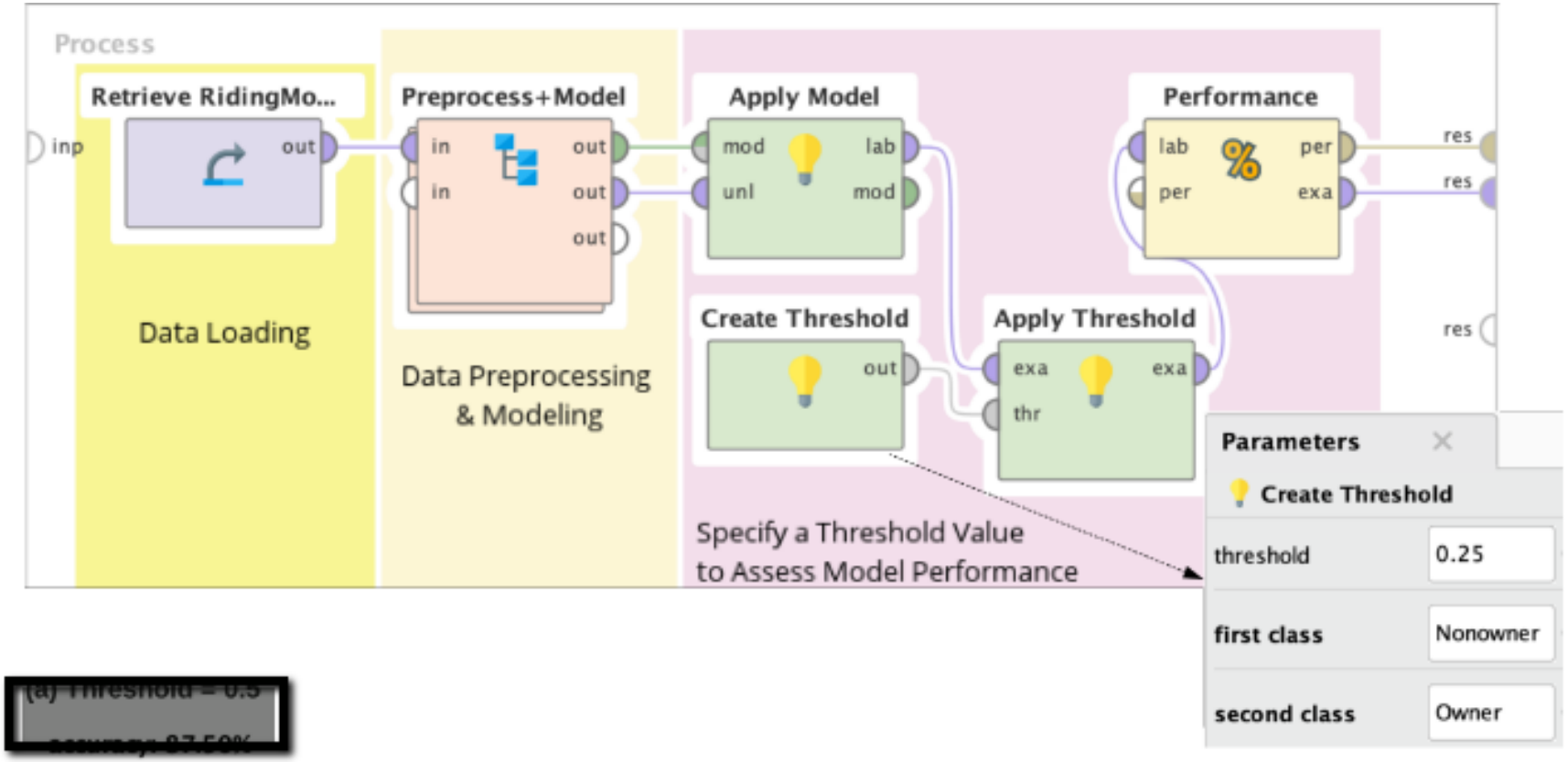
Most algorithms classify via a 2-step process:

For each record,

1. Compute **probability of belonging to class "1"**
2. Compare to threshold value, and classify accordingly

- Default threshold value is 0.50
  - If >= 0.50, classify as "1"
  - If < 0.50, classify as "0"
- Can use different threshold values
- Typically, threshold = 0.50

RapidMiner Process



(a) Threshold = 0.5

Threshold Table

Predicted Probability of Being an Owner

ACTUAL	Pred. Prob.	ACTUAL	Pred. Prob.
owner	0.9959	owner	0.5055
owner	0.9875	nonowner	0.4713
owner	0.9844	nonowner	0.3371
owner	0.9804	owner	0.2179
owner	0.9481	nonowner	0.1992
owner	0.8892	nonowner	0.1494
owner	0.8476	nonowner	0.0479
nonowner	0.7628	nonowner	0.0383
owner	0.7069	nonowner	0.0248
owner	0.6807	nonowner	0.0218
owner	0.6563	nonowner	0.0161
nonowner	0.6224	nonowner	0.0031



if cutoff is 0.5, then 11 records classified as “owner”

Threshold Table

Predicted Probability of Being an Owner

ACTUAL	Pred. Prob.	ACTUAL	Pred. Prob.
owner	0.9959	owner	0.5055
owner	0.9875	nonowner	0.4713
owner	0.9844	nonowner	0.3371
owner	0.9804	owner	0.2179
owner	0.9481	nonowner	0.1992
owner	0.8892	nonowner	0.1494
owner	0.8476	nonowner	0.0479
nonowner	0.7628	nonowner	0.0383
owner	0.7069	nonowner	0.0248
owner	0.6807	nonowner	0.0218
owner	0.6563	nonowner	0.0161
nonowner	0.6224	nonowner	0.0031



if cutoff is 0.8, then 7 records classified as "owner"

Confusion Matrix for Different Thresholds

cutoff = 0.5 (Accuracy 0.8750)

Actual	Prediction nonowner	owner
nonowner	10	2
owner	1	11

cutoff = 0.25 (Accuracy 0.7917)

Actual	Prediction nonowner	owner
nonowner	8	4
owner	1	11

Cutoff 0.75 (Accuracy 0.7500)

Actual	Prediction nonowner	owner
nonowner	11	1
owner	5	7

Accuracy

- **Accuracy:** Ratio of correctly classified cases (TP + TN) to total cases.
- **Advantage:**
  - Easy to understand,
  - Most widely-used metric,
  - Interpretable by non-experts.
- **Disadvantage:**
  - Ignores imbalances in class distribution,
  - Might not be suitable for skewed datasets.
  - Useful as a general indicator, however shouldn't be used alone.

Alternate Accuracy Measures: When One Class is More Important

In many cases it is more important to identify members of one class (important class)

- Response to promotional offer
- Detecting electronic network intrusion
- Predicting delayed flights
- Tax fraud
- Credit default

In such cases, we are willing to tolerate greater overall error, in return for better identifying the important class for further attention

Precision and Recall of a Positive Class

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	a (TP)	b (FN)
	Class=No	c (FP)	d (TN)

$$\text{Precision} = \frac{a}{a+c} = \frac{TP}{TP+FP}$$

$$\text{Recall} = \frac{a}{a+b} = \frac{TP}{TP+FN}$$

Precision and Recall of a Class

- **Precision:**
  - Ratio of correctly predicted positives (TP) to all predicted positives (TP + FP).
- **Advantage:**
  - Focuses on the model's ability to avoid false positives,
  - Crucial when false positives are costly (e.g., spam filtering).

- **Recall:**
  - Ratio of correctly predicted positives (TP) to all actual positives (TP + FN).
- **Advantage:**
  - Focuses on the model's ability to identify all true positives,
  - Crucial when missing true positives is unacceptable (e.g., medical diagnosis).
  - Important for high-cost-of-false-negative scenarios.

Metrics for Performance Evaluation

$$\text{Precision (p)} = \frac{a}{a+c}$$

$$\text{Recall (r)} = \frac{a}{a+b}$$

$$\text{F - measure (F)} = \frac{2rp}{r+p} = \frac{2a}{2a+b+c}$$

F1-Score

- **F1-Score:**
  - Harmonic mean of precision and recall, combining their information into a single metric.
  - A balanced option for imbalanced datasets or when both precision and recall are important.
- **Advantage:**
  - Balances the concerns of precision and recall, useful for evaluating models on imbalanced datasets.